Rencana Pembuatan Web

https://gemini.google.com/app/e365e255e17a2920

User prompt: saya mau buat web tempat untuk belajar coding, tapi siapkan dulu plannya

Response: Ini adalah rencana strategis komprehensif untuk membangun platform *e-learning* pemrograman. Rencana ini dibagi menjadi lima fase utama, dari konseptualisasi hingga peluncuran.

1. Definisi Konsep & Target Audiens

Sebelum menulis kode pertama, arah platform harus ditentukan dengan jelas agar fitur yang dibangun tepat sasaran.

- **Target Pengguna:** Apakah untuk pemula (anak sekolah/mahasiswa), *career switcher* (profesional yang ingin pindah jalur IT), atau *developer* berpengalaman yang ingin *upskill*?
- **Model Monetisasi:** Gratis sepenuhnya (donasi/open source), berlangganan (*subscription* bulanan/tahunan), atau bayar per kelas (*pay-per-course*).
- **Fokus Kurikulum:** Web Development (HTML, CSS, JavaScript, React), Data Science (Python, SQL), atau Mobile App Development (Flutter, Kotlin).

2. Rancang Bangun Fitur (MVP - Minimum Viable Product)

Untuk versi pertama, fokus pada fitur-fitur esensial yang memungkinkan pengguna mendaftar, belajar, dan mempraktikkan kode.

Kategori Fitur	Deskripsi	Prioritas
Sistem Autentikasi	Register/Login (Email, Google, GitHub OAuth).	Tinggi
Katalog Kelas	Daftar silabus, modul video/teks, dan sistem pencarian.	Tinggi
Interactive Code Editor	Teks editor di dalam browser (seperti VS Code) untuk langsung menulis dan menjalankan kode tanpa perlu instalasi di komputer pengguna.	Sangat Tinggi
Progress Tracking	Indikator persentase penyelesaian kelas dan *gamification* (sertifikat, *badge*, atau poin).	Menengah
Sistem Pembayaran	Integrasi *payment gateway* (Midtrans, Xendit, atau Stripe) jika berbayar.	Tinggi (Jika berbayar)

3. Rekomendasi Tech Stack

Memilih teknologi yang tepat sangat penting, terutama untuk menangani eksekusi kode pengguna secara aman.

- **Frontend (Antarmuka Pengguna):**

- **Next.js (React)** atau **Vue.js**: Sangat baik untuk platform edukasi karena performa SEO yang tinggi dan ekosistem UI yang kaya.
- *Library Tambahan:* Monaco Editor (komponen inti di balik VS Code) untuk membuat *interactive code editor* di browser.

- **Backend (Logika & API):**
  - **Node.js (Express/NestJS)** atau **Python (Django/FastAPI)**. Python sangat disarankan jika platform akan banyak berfokus pada eksekusi kode dan AI.
- **Database:**
  - **PostgreSQL** (Relasional) untuk data pengguna, transaksi, dan kemajuan belajar.
- **Code Execution Engine (Penting):**
  - Untuk menjalankan kode pengguna dengan aman, Anda membutuhkan lingkungan yang terisolasi (*sandboxed*). Gunakan **Docker containers** atau API pihak ketiga seperti **Judge0** / **JDoodle**.

4. Timeline Pengembangan (Estimasi 3-4 Bulan)

1. **Bulan 1: Perencanaan & Desain (UI/UX)**
   - Pembuatan *wireframe* dan *mockup* antarmuka (Figma).
   - Desain arsitektur database.
   - Pembuatan silabus untuk 1-2 kelas pertama sebagai purwarupa.
2. **Bulan 2: Pengembangan Core System**
   - Setup Frontend & Backend.
   - Pembuatan sistem autentikasi dan manajemen profil.
   - Pembuatan sistem manajemen konten (CMS) agar admin bisa mengunggah materi pelajaran.
3. **Bulan 3: Fitur Spesifik & Integrasi**
   - Integrasi *Interactive Code Editor* (Monaco Editor + Judge0/Docker).
   - Integrasi *Payment Gateway*.
   - *Quality Assurance* (QA) dan pengujian keamanan (terutama mencegah pengguna menjalankan kode berbahaya di server Anda).
4. **Bulan 4: Beta Testing & Peluncuran**
   - Rilis versi Beta ke sekelompok kecil pengguna untuk mencari *bug*.
   - Perbaikan berdasarkan umpan balik (*feedback*).
   - Peluncuran publik versi 1.0.

Agar kita bisa mematangkan rencana ini, bahasa pemrograman atau topik spesifik apa yang ingin Anda jadikan materi pertama di platform ini?

User prompt: ditujukan untuk pemula

Response: Menargetkan pemula adalah keputusan strategis yang sangat baik, namun ini menuntut pendekatan antarmuka dan pedagogi yang spesifik. Pemula sangat mudah merasa terintimidasi oleh layar kosong atau pesan *error* teknis.

Berikut adalah penyesuaian rencana platform yang dioptimalkan khusus untuk pemula:

1. Rekomendasi Alur Belajar Pertama

Pilih bahasa pemrograman yang memberikan *quick wins* (hasil yang bisa langsung dilihat) atau memiliki sintaks yang ramah.

- **Jalur Frontend Web (Paling Direkomendasikan):** Dimulai dari HTML dasar, lalu CSS, dan sedikit JavaScript. Pemula sangat menyukai jalur ini karena mereka langsung melihat teks berubah menjadi tombol atau halaman web yang berwarna.
- **Jalur Logika & Data (Alternatif):** Python. Sintaksnya paling mirip dengan bahasa Inggris manusia. Fokus pada logika dasar (variabel, *looping*, *if/else*) tanpa perlu memikirkan struktur kode yang rumit seperti pada Java atau C++.

2. Penyesuaian Fitur (UI/UX)

- **Micro-learning:** Pecah materi menjadi sangat kecil (contoh: 1 video maksimal 3 menit atau 1 halaman teori singkat), langsung diikuti oleh 1 tantangan koding.
- **Scaffolding (Bantuan Bertahap):** Jangan langsung berikan *code editor* kosong. Pada tahap awal, gunakan format "Isi Bagian Rumpang" (*fill-in-the-blanks*). Setelah mereka paham, barulah berikan tugas menulis kode dari nol.
- **Penerjemah Pesan Error:** Ini fitur krusial untuk pemula. Jangan tampilkan teks *error* merah bawaan *compiler* (seperti `SyntaxError: unexpected token`). Ubah menjadi pesan yang ramah, contoh: *"Ups! Sepertinya kamu lupa menutup tanda kurung di baris 4."*
- **Gamifikasi Visual:** Berikan animasi perayaan (seperti efek *confetti*) setiap kali mereka berhasil menjalankan kode, serta *streak* harian untuk menjaga konsistensi belajar.

3. Strategi Akuisisi Pengguna

Pemula biasanya belum mau berkomitmen membayar sebelum yakin mereka bisa mengerti materinya.

- Gunakan model **Freemium**. Berikan kelas pengenalan (misal: "Basic HTML" atau "Hello World in Python") secara 100% gratis.
- Kunci (*lock*) materi tingkat lanjut atau proyek akhir, dan berikan opsi pembayaran sekali putus (*pay-per-course*) yang terjangkau untuk kantong pelajar/mahasiswa.

Apakah Anda lebih tertarik untuk mengembangkan kurikulum dan materi pertamanya di jalur **Web (HTML/CSS)** atau jalur **Python**?

User prompt: gunakan bahasa dasar terlebih dahulu agar mempunyai dasar. tetap memasukan pembalajaran dalam bentuk teori serta bisa menayangkan video

Response: Pilihan untuk menanamkan pondasi dasar (algoritma dan logika) dipadukan dengan teori dan video adalah pendekatan edukasi yang sangat solid. Pendekatan ini menyerupai kurikulum Ilmu Komputer formal namun dikemas secara modern.

Berikut adalah cetak biru (*blueprint*) platform yang disesuaikan untuk kebutuhan pembelajaran dasar yang menggabungkan video, teori, dan praktik:

1. Kurikulum "Pondasi Pemrograman"

Jika fokusnya adalah "bahasa dasar", bahasa **Python** atau **C++** adalah pilihan terbaik. Python unggul untuk melatih logika tanpa pusing dengan sintaks, sementara C++ sangat baik untuk mengajarkan bagaimana komputer bekerja di belakang layar (manajemen memori).

Silabus dasar bisa dibagi menjadi:

- **Teori Komputasi Dasar:** Apa itu program? Bagaimana komputer membaca kode?
- **Konsep Inti:** Variabel, Tipe Data, dan Operator.
- **Struktur Kontrol:** Logika percabangan (*If/Else*) dan Perulangan (*For/While loops*).
- **Fungsi & Modularitas:** Cara memecah kode menjadi blok-blok yang bisa digunakan ulang.
- **Struktur Data Dasar:** *Array/List* dan *Dictionary*.

2. Desain Layar Pembelajaran (*Split-Screen UI*)

Karena Anda membutuhkan video, teori, dan praktik sekaligus, antarmuka (*User Interface*) terbaik adalah model **layar terbagi (Split-Screen)**.

- **Sisi Kiri (Materi):**
    - **Tab 1 - Video:** *Video player* untuk penjelasan visual/animasi konsep.
    - **Tab 2 - Teori (Teks):** Penjelasan tertulis (menggunakan format *Markdown*) agar pengguna bisa membaca ulang konsep tanpa harus memutar video dari awal.
    - **Tab 3 - Kuis Teori:** Pertanyaan pilihan ganda untuk memastikan mereka paham teori sebelum mulai mengoding.
- **Sisi Kanan (Praktik):**
    - **Area Kode:** Tempat pengguna mengetik kode.
    - **Area Terminal/Console:** Tempat menampilkan hasil (*output*) dari kode yang mereka jalankan.

3. Penyesuaian Infrastruktur Teknologi (*Tech Stack*)

Menambahkan fitur video membutuhkan strategi *hosting* khusus agar *server* Anda tidak *down* karena kehabisan *bandwidth*.

- **Video Hosting: Jangan simpan video di server database Anda sendiri.** Gunakan layanan pihak ketiga seperti **Vimeo (paket bisnis untuk menyembunyikan video dari publik)**, **Mux**, atau **AWS S3**. Anda hanya perlu menautkan URL videonya ke dalam platform Anda.
- **Content Management System (CMS):** Buat *dashboard admin* menggunakan *Headless CMS* (seperti Strapi atau Sanity) agar instruktur bisa dengan mudah:
  - Menempelkan tautan video.
  - Menulis materi teori menggunakan *Rich Text Editor* atau *Markdown*.
  - Menyiapkan kunci jawaban untuk soal koding.
- **State Management Video:** Fitur tambahan di *frontend* agar video bisa otomatis berhenti (*auto-pause*) ketika pengguna berpindah ke tab *Code Editor*.

4. Alur Belajar Pengguna (*User Journey*)

Untuk memastikan pemula benar-benar paham, buat alur paksaan (*gated progression*) per modul:

1. **Tonton:** Pengguna wajib menonton video teori berdurasi 3-5 menit.
2. **Baca & Jawab:** Pengguna membaca ringkasan teori dan menjawab 2 soal pilihan ganda.
3. **Praktik Terbimbing:** Pengguna menulis kode berdasarkan teori yang baru saja dipelajari. Jika kode salah, sistem memberikan petunjuk (bukan jawaban langsung).
4. **Lulus:** Modul berikutnya akan terbuka hanya jika kode berhasil dijalankan (*Passed*).

Untuk langkah selanjutnya, apakah Anda ingin mengelola pembuatan materi (video dan silabus) ini sendiri, atau berencana mengundang kreator/instruktur lain untuk mengunggah materi mereka ke web Anda (*marketplace model*)?

User prompt: siapkan kurkulumnya

Response: Kurikulum Pondasi Pemrograman (Berbasis Python)

Python dipilih sebagai bahasa pengantar karena sintaksnya sangat intuitif bagi pemula, memungkinkan pengguna fokus pada logika pemrograman daripada menghafal aturan penulisan kode yang rumit. Kurikulum ini dirancang untuk alur *Split-Screen* (Video -> Teori -> Praktik).

Modul 1: Berkenalan dengan Dunia Kode

Fokus: Memberikan kemenangan pertama (*quick win*) agar pengguna termotivasi.

- **Video (3 Menit):** Cara kerja komputer membaca instruksi (Input -> Proses -> Output).
- **Teori:** Pengenalan bahasa Python dan fungsi `print()`.
- **Tantangan Praktik:** Menampilkan teks "Hello World!" dan nama pengguna ke layar terminal.

Modul 2: Kotak Penyimpanan (Variabel & Tipe Data)

Fokus: Memahami bagaimana data disimpan di dalam memori komputer.

- **Video (4 Menit):** Analogi variabel sebagai kotak berlabel yang menyimpan barang.
- **Teori:** Aturan penamaan variabel dan tipe data dasar (*String/Teks, Integer/Angka, Float/Desimal, Boolean/Benar-Salah*).
- **Tantangan Praktik:** Membuat form biodata diri sederhana menggunakan variabel dan menampilkannya dalam format kalimat utuh.

Modul 3: Otak Komputer (Operator & Kalkulasi)

Fokus: Melakukan manipulasi data dasar menggunakan matematika.

- **Video (3 Menit):** Bagaimana komputer berhitung jauh lebih cepat dari manusia.
- **Teori:** Operator aritmatika (+, −, ∗, /) dan Modulo (% - sisa bagi).
- **Tantangan Praktik:** Membuat kalkulator sederhana untuk menghitung luas persegi panjang berdasarkan *input* panjang dan lebar dari pengguna.

Modul 4: Membuat Keputusan (Percabangan / *If-Else*)

Fokus: Melatih logika sebab-akibat.

- **Video (5 Menit):** Analogi persimpangan jalan berdasarkan kondisi tertentu.
- **Teori:** Operator perbandingan (==, >, <) dan struktur `if`, `elif`, `else`.
- **Tantangan Praktik:** Membuat program penyeleksi umur (misal: menentukan apakah pengguna sudah cukup umur untuk membuat KTP) atau program penentu nilai rapor (A, B, C, D).

Modul 5: Bekerja Tanpa Lelah (Perulangan / *Loops*)

Fokus: Otomatisasi tugas yang berulang-ulang.

- **Video (4 Menit):** Konsep perulangan dan bahaya *infinite loop* (perulangan tanpa henti).
- **Teori:** `For loop` (untuk jumlah yang diketahui) dan `While loop` (berdasarkan kondisi).
- **Tantangan Praktik:** Membuat program roket hitung mundur dari 10 ke 1, diakhiri dengan teks "Meluncur!".

Modul 6: Rak Buku Komputer (Struktur Data - *List*)

Fokus: Menyimpan lebih dari satu data dalam satu variabel.

- **Video (4 Menit):** Analogi rak sepatu berurutan untuk memahami indeks.
- **Teori:** Cara membuat *List*, menambah data (`append`), menghapus data (`remove`), dan memanggil data berdasarkan indeks (dimulai dari 0).
- **Tantangan Praktik:** Membuat aplikasi "Daftar Belanja Harian", di mana pengguna bisa menambah dan melihat isi daftar belanjaan mereka.

Modul 7: Membuat Alat Sendiri (Fungsi / *Functions*)

Fokus: Modularitas dan kebersihan kode (*clean code*).

- **Video (5 Menit):** Analogi fungsi sebagai mesin pembuat kopi (masukkan biji kopi & air -> keluar secangkir kopi).
- **Teori:** Mendefinisikan fungsi (`def`), memberikan parameter, dan mengembalikan nilai (`return`).
- **Tantangan Praktik:** Memecah kalkulator dari Modul 3 menjadi fungsi-fungsi terpisah untuk tambah, kurang, kali, dan bagi.

Modul 8: Proyek Akhir (*Capstone Project*)

Fokus: Menguji semua konsep yang telah dipelajari dalam satu program utuh.

- **Instruksi (Teks & Video):** Penjelasan aturan proyek akhir tanpa panduan kode spesifik (menguji kemandirian).
- **Tantangan Praktik (Pilih Salah Satu):**
    - **Opsi A:** Membuat *Game* Batu-Gunting-Kertas melawan komputer (menggunakan modul `random`).
    - **Opsi B:** Membuat aplikasi "To-Do List" interaktif di mana pengguna bisa mengetik perintah untuk menambah, menghapus, atau melihat daftar tugas.

Bagaimana tanggapan Anda mengenai struktur silabus ini? Jika sudah sesuai, kita bisa membahas tahap selanjutnya yaitu penentuan detail *framework* teknologi untuk membangun webnya.

User prompt: pertemua di lakukan di hari jumat dan sabtu, dan setiap harinya dari hari senin sampai kami disiapkan tugas harian atau bisa disebut juga pr

Response: Format kombinasi antara kelas tatap muka (*live session*) di akhir pekan dan tugas mandiri (*asynchronous*) di hari kerja adalah model *bootcamp* yang sangat efektif. Model ini membangun kebiasaan *ngoding* setiap hari tanpa membebani jadwal harian pengguna.

Ini adalah struktur jadwal mingguan dan pemetaan kurikulum menjadi program 4 Minggu (1 Bulan).

Pola Jadwal Mingguan Standar

Sistem web Anda perlu dirancang untuk merilis (*unlock*) tugas harian secara bertahap agar pengguna tidak mengerjakan semuanya sekaligus.

Hari	Jenis Kegiatan	Fokus Aktivitas di Platform
Senin	**PR 1: *Recall***	Kuis pilihan ganda atau isi-bagian-rumpang (*fill-in-the-blanks*) untuk mengulang materi akhir pekan lalu.
Selasa	**PR 2: *Code Reading***	Membaca kode yang sudah jadi dan menebak apa hasil (*output*)-nya. Ini melatih nalar tanpa harus mengetik.
Rabu	**PR 3: *Debugging***	Diberikan kode yang *error* (salah ketik/logika). Tugas pengguna adalah memperbaiki (*fix the bug*) agar bisa berjalan.
Kamis	**PR 4: *Pre-read/Mini-Task***	Membaca sedikit teori pengantar untuk materi hari Jumat, ditambah 1 latihan koding ringan.
Jumat	**Pertemuan 1 (*Live*)**	Pembahasan materi baru bagian pertama + Praktik bersama instruktur.
Sabtu	**Pertemuan 2 (*Live*)**	Pembahasan materi baru bagian kedua + Sesi bedah kasus/Tanya jawab.
Minggu	**Istirahat**	Platform menghentikan notifikasi agar pengguna tidak *burnout*.

Pemetaan Kurikulum (Program 1 Bulan)

Kita distribusikan 8 Modul sebelumnya ke dalam format jadwal ini.

Minggu 1: Berkenalan dengan Python & Memori

- **Senin - Kamis (PR):** Instalasi *tools* (jika menggunakan editor lokal) atau pemanasan menggunakan *code editor* di web Anda. Latihan mencetak teks dengan `print()` dan bereksperimen dengan nama variabel.
- **Jumat (Pertemuan 1):** Modul 1 - Konsep dasar program dan eksekusi kode.
- **Sabtu (Pertemuan 2):** Modul 2 - Variabel dan Tipe Data dasar.

Minggu 2: Matematika & Pengambilan Keputusan

- **Senin - Kamis (PR):** Memperbaiki kalkulator yang salah hitung (*debugging*), latihan mengubah tipe data (teks ke angka), dan kuis logika perbandingan.
- **Jumat (Pertemuan 1):** Modul 3 - Operator Aritmatika.
- **Sabtu (Pertemuan 2):** Modul 4 - Logika Percabangan (*If-Elif-Else*).

Minggu 3: Mengotomatisasi Pekerjaan & Menyimpan Data Banyak

- **Senin - Kamis (PR):** Latihan logika bersyarat berlapis (*nested if*), memprediksi jumlah perulangan dari sebuah kode, dan manipulasi teks.
- **Jumat (Pertemuan 1):** Modul 5 - Perulangan (*For & While Loops*).
- **Sabtu (Pertemuan 2):** Modul 6 - Struktur Data (*List*).

Minggu 4: Kerapian Kode & Eksekusi Proyek

- **Senin - Kamis (PR):** Latihan menambah/menghapus isi *List*, menggabungkan *List* dengan perulangan, dan menyusun kerangka algoritma untuk proyek akhir.
- **Jumat (Pertemuan 1):** Modul 7 - Pembuatan Fungsi (*Functions & Modular Code*).
- **Sabtu (Pertemuan 2):** Modul 8 - *Live Coding Capstone Project* (mengerjakan proyek akhir bersama-sama atau presentasi hasil).

Untuk memfasilitasi PR harian ini, apakah Anda ingin sistem webnya nanti bisa memeriksa jawaban kode pengguna secara otomatis (*Auto-Grader*), atau PR tersebut akan diperiksa secara manual oleh instruktur/mentor pada saat pertemuan hari Jumat?

User prompt: siapkan hingga bulan desember

Response: Mengingat saat ini kita berada di akhir Oktober 2026, kita memiliki waktu efektif sekitar **8 minggu (2 Bulan)** untuk merancang kurikulum hingga akhir Desember.

Jika Bulan 1 (Fase Pondasi) sudah diselesaikan, kita akan merancang **Bulan 2 (November)** untuk pendalaman materi menengah, dan **Bulan 3 (Desember)** untuk penerapan ke dunia nyata serta Proyek Kelulusan (*Demo Day*).

Tetap menggunakan format **Senin-Kamis (PR Harian)** dan **Jumat-Sabtu (Kelas Live)**, berikut adalah rancangan kurikulum lengkap hingga akhir tahun:

Bulan 2: November (Fase Menengah & Pemecahan Masalah)

Fokus bulan ini adalah membuat program yang lebih kompleks, bisa menyimpan data secara permanen, dan tidak mudah *crash* saat terjadi kesalahan.

Minggu 5: Laci Pintar (Dictionary & Tuples)

- **Senin - Kamis (PR):** Mengakses data menggunakan *key* (kunci), merapikan data JSON sederhana, dan kuis perbedaan *List* vs *Dictionary*.
- **Jumat (Live):** Memahami *Dictionary* (menyimpan data dengan format kunci-nilai, seperti kamus) dan *Tuple* (data yang tidak bisa diubah).
- **Sabtu (Live):** Latihan membuat sistem "Keranjang Belanja" lengkap dengan harga dan jumlah barang.

Minggu 6: Anti-Crash & Menyimpan File (*Error Handling & File I/O*)

- **Senin - Kamis (PR):** *Debugging* kode yang memunculkan pesan *error*, latihan membaca isi file `.txt` sederhana, dan tugas menulis nama pengguna ke dalam file.

- **Jumat (Live):** Menggunakan `try-except` agar program tidak tiba-tiba berhenti saat *error* (misal: pengguna memasukkan huruf saat diminta memasukkan angka).
- **Sabtu (Live):** Cara membuat program membaca dan menyimpan data secara permanen ke file `.txt` atau `.csv` (Microsoft Excel).

Minggu 7: Berpikir Seperti Arsitek (*Object-Oriented Programming / OOP* - Bagian 1)

- **Senin - Kamis (PR):** Membaca teori tentang "Objek" di dunia nyata, menebak atribut dari sebuah objek (misal: Mobil punya warna, roda, merk).
- **Jumat (Live):** Pengenalan *Class* dan *Object*. Mengubah gaya penulisan kode dari yang berantakan menjadi terstruktur.
- **Sabtu (Live):** Latihan membuat *Class* "Karakter Game" yang memiliki Nyawa (HP) dan Kekuatan Serang.

Minggu 8: Proyek Tengah Semester (Aplikasi Konsol)

- **Senin - Kamis (PR):** Menyusun kerangka sistem yang akan dibuat, memecah tugas menjadi fungsi-fungsi kecil.
- **Jumat (Live):** *Live Coding* bersama membuat **Aplikasi Buku Kontak** (Bisa tambah nama, nomor telepon, simpan ke file, dan cari kontak).
- **Sabtu (Live):** Evaluasi proyek, *code review* (bedah kode pengguna), dan sesi tanya jawab santai.

Bulan 3: Desember (Fase Implementasi Nyata & Kelulusan)

Di bulan terakhir ini, kita akan menghubungkan kode Python yang sudah dipelajari dengan dunia luar (internet dan tampilan visual), diakhiri dengan perayaan kelulusan sebelum libur akhir tahun.

Minggu 9: Menghubungkan Kode dengan Internet (API & JSON)

- **Senin - Kamis (PR):** Membaca teori tentang bagaimana web saling berkomunikasi (Konsep *Client & Server*), latihan mengambil data *dummy* dari internet.
- **Jumat (Live):** Pengenalan API (*Application Programming Interface*). Membaca data dari internet menggunakan pustaka `requests`.
- **Sabtu (Live):** Membuat program pemeriksa cuaca atau penampil kurs mata uang harian yang mengambil data langsung secara *real-time* dari internet.

Minggu 10: Tampilan Visual (*Web App* Sederhana) *(Catatan: Menggunakan pustaka seperti **Streamlit** agar pemula bisa menyulap kode Python menjadi web interaktif tanpa harus belajar HTML/CSS yang rumit terlebih dahulu).*

- **Senin - Kamis (PR):** Eksperimen dengan tombol, kolom *input* teks, dan menampilkan gambar di web.
- **Jumat (Live):** Mengubah program konverter mata uang atau kalkulator yang sebelumnya hitam-putih di terminal menjadi web interaktif.
- **Sabtu (Live):** *Brainstorming* ide untuk Proyek Akhir Kelulusan.

Minggu 11: Minggu Pengerjaan Proyek Akhir Kelulusan

- **Senin - Kamis (PR):** Tidak ada tugas baru. Pengguna wajib mengirimkan *update* (laporan progres) harian terkait proyek web interaktif yang mereka bangun sendiri.
- **Jumat (Live):** Sesi Konsultasi & *Debugging Clinic*. Instruktur membantu memperbaiki *error* atau kebuntuan (*stuck*) yang dialami peserta.
- **Sabtu (Live):** Finalisasi proyek dan persiapan bahan presentasi.

Minggu 12: *DEMO DAY* & Kelulusan (Tutup Tahun)

- **Senin - Kamis:** Pengumpulan proyek akhir, kode program, dan persiapan mental.
- **Jumat (Live): Demo Day Part 1.** Sebagian peserta mempresentasikan web app/program yang mereka buat, menceritakan kendala, dan apa yang dipelajari.
- **Sabtu (Live): Demo Day Part 2 & Wisuda Virtual.** Presentasi peserta tersisa, pengumuman peserta terbaik, pembagian Sertifikat Kelulusan digital, dan penutupan program bootcamp sebelum memasuki masa libur panjang Natal & Tahun Baru.

Dengan rencana ini, pada akhir bulan Desember, pengguna yang awalnya sama sekali tidak mengerti kode sudah memiliki portofolio berupa aplikasi web sederhana yang hidup dan bisa dipamerkan.

Apakah alur PR dan materi ini sudah pas dengan visi yang ingin Anda bangun untuk para pemula tersebut?
